

A Data-Analysis Agent, Built From Scratch

Part 1: two benchmark papers on autonomous scientific analysis

Part 2: a design built against what they found

Musel Tabares Pardo

July 2026

Part 1 — the papers

- Why measure this now — and what is even under test
- **GeneBench-Pro**: motivation, construction, grading, limits
- **DrugDiscoveryBench**: the same story, on real data
- **How these agents actually fail** — five failures

Part 2 — the design

- The assignment, and what an agent *is*
- **The architecture**: a loop, three ledgers, a gate
- **Each failure in Part 1 earns one mechanism**
- How I would know whether it worked — and a live demo

The through-line: **every mechanism in Part 2 is a named failure from Part 1.**

Part I — The papers

1. Context: can an agent do a scientist's analysis?
2. GeneBench-Pro — simulate, so you can grade judgement
3. DrugDiscoveryBench — real tasks, an expert's playbook
4. How these agents fail — and what each failure demands

Part II — The design

5. The design: an LLM, a loop, three ledgers, a gate
6. How would I know it works?

Part I

The papers

Part I — The papers

1. Context: can an agent do a scientist's analysis?
2. GeneBench-Pro — simulate, so you can grade judgement
3. DrugDiscoveryBench — real tasks, an expert's playbook
4. How these agents fail — and what each failure demands

Part II — The design

5. The design: an LLM, a loop, three ledgers, a gate
6. How would I know it works?

Why measure this now

AI agents can now write and run code in a loop, unsupervised. So the question both papers ask is the same: can they do the part of science that is judgement, not typing?

- **The premise.** Coding agents have improved fast. It is worth finding out, carefully, how well they do *real scientific research* — and the only way to know is a **benchmark that actually informs you**.
- **The bottleneck moved.** Sequencing got cheap; the analysis did not. “as sequencing has scaled, downstream computation and analysis, rather than data generation, have become the central scientific bottleneck.” — GBP, p. 2
- **The stakes are real.** Drug targets with human genetic evidence are “*roughly 2.6 times more likely to succeed in the clinic*”. — DDB, p. 2
- **And the step is slow and expensive.** One GBP problem is **10–40 hours** of expert time — “*on the order of a few thousand dollars*”. — GBP, p. 14

So the question is worth a benchmark

If an agent could reliably do this analysis, it would compress the slowest, most expensive step in data-rich biology. **Can it — and how would you ever know?**

What is actually under evaluation — and the two papers

*Neither paper evaluates a bare LLM. What is measured is a **model** + a **harness**, inside a container.*

What runs, per task

- ① A **Docker container**: Python, the scientific stack, the data files staged on disk.
 - ② A **harness** drives the loop — sends the prompt, **executes the code the model emits**, feeds stdout back. The model never touches the machine.
 - ③ The model gets the question **at a high level**, *not* the method: which rows to keep, which estimator, when the evidence suffices. **No human, no hints.**
 - ④ It emits **one machine-checkable answer** — GBP: a JSON object. DDB: `/workspace/answer.md`.
- **GeneBench-Pro** (Li & Ho, **OpenAI** — bioRxiv, June 2026): **129 problems**, genomics and translational biomedicine. **Graded by deterministic Python.**
 - **DrugDiscoveryBench** (Akyürek, Tu et al., **Scale AI & Phylo**, 2026): **82 tasks**, early-stage drug discovery. **Graded by an LLM judge** against expert rubrics.

Keep this in your pocket: “the model scored *X*” means “*in this harness, X*”

DDB measures it — one model in a different harness moves **8 points** (43.9% → 51.6%), more than the gap between the #1 and #2 *models*. Both are preprints: GBP “not certified by peer review”; DDB prints no venue.

Part I — The papers

1. Context: can an agent do a scientist's analysis?
2. GeneBench-Pro — simulate, so you can grade judgement
3. DrugDiscoveryBench — real tasks, an expert's playbook
4. How these agents fail — and what each failure demands

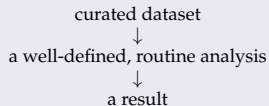
Part II — The design

5. The design: an LLM, a loop, three ledgers, a gate
6. How would I know it works?

GBP — the motivation: benchmarks weren't testing the hard part

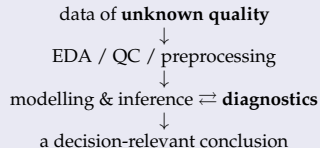
The paper exists to close a gap. Real scientific analysis is far more complex than prior benchmarks measure — and there are too few representative ones.

What prior benchmarks test



The method is *given*. This measures **execution**.

What real scientific analysis is



The method must be **chosen**. This measures **judgement**.

*“Its core challenges stem not from the execution of analytical workflows, but from the importance of scientific intuition or ‘**research taste**’: chains of judgment calls about what question the data can support, what data to include, which estimand or model is appropriate...”*

GeneBench-Pro, p. 2 — verbatim

Sample collection is no longer the limiting factor — **analysis is**. So a good benchmark measures the exact step an AI agent could most help with.

129 problems in genomics and translational biomedicine. But the unit they are built from is the thing to remember.

- **129 problems, 10 primary domains**, 21 terminal subdomains.
- **Fully simulated** data — the causal structure is *known*, and nothing can be memorised from the literature.
- Docker, **no internet**, one fixed harness, a deliberately minimal prompt.
- Staged files look like what an analyst receives: batch effects, sentinel values, missingness, mixed ancestry — the defects are *deliberate*, and they are the point.

The central construct — the *decision point*

“substantive inferential forks where a plausible wrong choice leads to a qualitatively different downstream answer”

p. 7 — 3–13 per problem, median 6

~Six places, per problem, to take a defensible-looking wrong turn

And the errors **cascade**: an incorrect choice at any stage propagates downstream and the agent never recovers the correct target. **This is the idea I took wholesale for Part 2**: plant the fork, know where the wrong path lands — then you can say not just *that* the agent was wrong, but **which wrong**.

GBP — the justification: build it synthetically, for full control

The one design decision everything clever about GBP falls out of. It is how they avoid the two ways a long-horizon benchmark silently breaks.

Two ways these benchmarks break

- 1. Too many defensible paths.** Multi-step questions built on *messy historical data* have no single correct route. One analyst's cutoff is as defensible as another's — so the score reflects the **author's arbitrary preference**, not model skill.
- 2. Too numerically insensitive.** If the answer barely moves, an agent can make a **fundamental error** and still pass.

GBP's answer: simulate the data-generating process

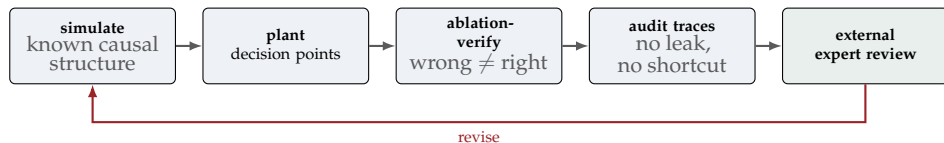
Because they **know the full causal structure**, they can:

- **tune** the complexity of each problem;
- ensure **reasonable subjective choices still pass**;
- **verify by ablation** that plausible-but-wrong analyses *fail*;
- **audit traces** for information leakage and unintended shortcuts.

Getting the right answer depends on the right analytic *pathway* — not on matching an author's preference, and not on exploiting a shortcut. (GBP §2, paraphrased; principles in their Table 1.)

GBP — construction & validation: the build-and-review workflow

How a single problem is built, checked, and signed off. This is the “image” worth showing.



The external review is the part you cannot generate

11 domain experts — *graduate students, postdoctoral researchers, industry scientists, and professors*. **84 candidate problems reviewed, 82 kept** in the suite. Each reviewer assessed:

is it realistic? is the target answer identifiable? are the methods and estimators appropriate?

Feedback drove **prompt/data fixes** and the **redesign** of problems whose targets were not sufficiently identifiable.

GeneBench-Pro, pp. 7–8 — “External Scientific Review”

GBP — grading: the subtlest choice in either paper

Easy to skim past. It is the difference between a benchmark that works and one that is silently broken.

You simulated the data, so you know the true parameter you dialled in. **Grade against that? No.**

The naive way (broken)

Simulator knob: true effect $\beta = 0.30$. But the staged files are a *finite sample*; in *those files* a **flawless** analyst recovers $\hat{\beta} = 0.283$ — sampling noise.

Grade against 0.30 ± 0.01 : **a perfect analysis is marked WRONG.**

What GBP does (right)

Grade against **the estimate a correct analysis can actually recover from the staged files** — the **recoverable target**, *not* the hidden generating parameter.

Now the grader tests the *analysis*, not the dice.

And the grade itself is **binary, deterministic Python** — *no LLM judge anywhere*

*“an agent that executes several intermediate steps correctly but returns the wrong decision-relevant answer **has not successfully automated the analysis.**” (p. 14)*

Partial credit measures effort. Binary measures usefulness.

GBP — the results are brutal

The last column is the one to look at — and it is not the headline.

Configuration	Pass rate	95% CI	Problems never solved (0% pass, all attempts)
GPT-5.6 Sol Pro (Extended) — <i>best in paper</i>	31.5%	—	
GPT-5.6 Sol (max) — <i>best mainline</i>	28.7%	[22.5, 35.1]	45.7%
GPT-5.6 Terra (max)	23.3%	—	
Claude Opus 4.8 (max) — <i>strongest non-GPT</i>	16.0%	—	
GPT-5.5 (xhigh)	12.0%	—	
Non-GPT range across all rows: 0.6% (MiniMax M2.7) to 16.0% (Claude Opus 4.8)			

Body text pp. 3–10 and Supplementary Table 1. The never-solved share is separate, not derivable from the pass rate.

The number that should stop you — and it is not 28.7%

28.7% is an *average over problems*. The best mainline model scores **zero across all ten attempts on 45.7% of problems** — nearly half the benchmark is **never solved once, in ten tries**. Reasoning effort buys a lot then saturates: GPT-5.6 Sol, *none*→*max*, runs 3.7 → 14.4 → 22.5 → 24.4 → 26.8 → 28.7 — an 8× gain, but *high*→*max* is only +1.9.

Two, and both bound how far you can read the numbers.

- **OpenAI is evaluating OpenAI's own models.** The leaderboard is topped by GPT-5.6 (Sol / Pro), authored by the same lab. Judge-free grading protects the *scores*, and they argue hardening tasks *with* GPT biases *against* it — fair. But the **choice of headline model and the framing are in-house**.
- **No human baseline.** There is a *time* reference (10–40 expert-hours per problem), but **no human accuracy to read 28.7% against**. Is an expert 90%? 60%? The paper does not say.

So what the numbers actually tell you

Not “how close to a scientist” — there is no scientist in the table. Only **how good these models are at these particular tasks**, relative to each other. That is still a great deal, but it is a narrower claim than the headline invites.

Part I — The papers

1. Context: can an agent do a scientist's analysis?
2. GeneBench-Pro — simulate, so you can grade judgement
3. DrugDiscoveryBench — real tasks, an expert's playbook
4. How these agents fail — and what each failure demands

Part II — The design

5. The design: an LLM, a loop, three ledgers, a gate
6. How would I know it works?

DDB — what it measures, and the bet it makes

The second paper trades GBP's simulated certainty for real-world grounding. Same story, opposite construction choice.

Early-stage drug discovery is a long, high-attrition funnel

target ID → hit ID → hit-to-lead → lead optimisation

DDB measures how reliably frontier **coding agents** do the multi-step *computational* and *information-retrieval* work of this pipeline. It **stops before candidate selection** — DMPK, toxicology, formulation and clinical work are out of scope.

DDB's bet: pay experts to *author* truth on real data

82 tasks written by pharmaceutical scientists, each anchored to **one real artifact** — a granted patent, a PDB structure, a published paper, or a database record. Each resolves to a **single verifiable answer** — an integer, a float, a SMILES/InChIKey, a ranking, or a short table.

Where GBP *simulates* to own the truth, DDB *pays experts* to author it.

Expert-authored tasks	82
Research capabilities covered	7
Lifecycle stages spanned	4 (+adjacent)
Total weighted rubric criteria	920
Biomni functions / domains	226 / 22
Data-lake files	76
Preinstalled scientific packages	117

DrugDiscoveryBench, Table 1

A representative task (lead optimisation)

Two KRAS G12C co-crystal structures (6USX, 6UT0). *Why is compound B more potent?*

The agent must fetch both PDB entries, superimpose them, find the residue whose conformation shifts, and confirm it contacts compound B. **Answer: THR58.**

The contrast that will matter

GBP: no internet, plain scientific stack. **DDB:** internet on, a 226-function domain toolbox, 120 min/task.
More realism, more capability — and, as we will see, more ways for the evaluation to leak.

DDB — construction: the authoring pipeline

Six gates. Gate 4 is the one to remember — it decides which tasks are allowed into the benchmark.



Gates 4–5 together are an *inclusion criterion* — hold on to this

A task enters the benchmark only if an agent **can** solve it *when handed the expert's playbook*; tasks that cannot be made robust are **dropped**. It quietly determines what the paper's headline finding can possibly show — and it comes back in the limitations.

How a real-data, open-ended answer gets scored — and the experiment that isolates planning from execution.

- **Presentation — Harbor format.** A task prompt + a shared footer pointing at the on-container references, the data lake, and the requirement to write only the final answer to `/workspace/answer.md`.
- **Harnesses.** Native where available (**Codex, Claude Code, Gemini CLI**); **mini-SWE-agent** as the universal cross-model fallback.
- **Reasoning effort.** Swept low / medium / ceiling (xhigh for GPT, max for Opus, high for Gemini). Every cell is **3 runs averaged**, 120-min timeout.
- **Grading.** An LLM judge (**GPT-5.4**) scores `answer.md` against the author's **weighted rubric**; **pass = 100% of the outcome criteria**. Cross-checked vs Sonnet 4.6 and Gemini 3.5 Flash: $\kappa = 1.0$ on binary labels.

Expert-method recovery — planning vs execution

Give three frontier agents the expert's **playbook** (the step sequence and which tools — *never* the answer). **76 of 82** tasks already fall with no hints (best-of-many); the playbook then lifts the remaining unsolved to **80 of 82 (+1 near-pass)**.

“execution is within reach for today’s agents should they be given the expert workflow.” (p. 14)

Setting	Harness	Pass rate
GPT-5.5 (xhigh) — <i>best</i>	mini-SWE-agent	51.6%
Gemini 3.5 Flash (high)	Gemini CLI	50.0%
Claude Opus 4.8 (max)	Claude Code	45.1%
GPT-5.5 (xhigh)	Codex (<i>native</i>)	43.9%
Claude Opus 4.6 (max) / Sonnet 4.6 (max)	Claude Code	27.7% / 24.0%
MiniMax M3 (xhigh) — <i>bottom</i>	mini-SWE-agent	23.2%

DrugDiscoveryBench, §4 and the leaderboard figures. Best model solves *just over half*.

Three findings worth the price of admission

- 1. Test-time compute scales** — within a family. GPT-5.5 (Codex): 27.6 → 39.8 → 43.9% (~ 18 pts for $4\text{--}5\times$ the tokens; one generation of frontier progress $\approx 8\text{--}20$ pts).
- 2. The harness matters as much as the model.** GPT-5.5 gains **+7.7 pts** moving from its native Codex (43.9%) to mini-SWE-agent (51.6%) — larger than the #1–#2 *model* gap.
- 3. What separates models is *unguided high-level planning*** — the recovery experiment flips the unsolved tasks once the plan is supplied. The gap is planning, not execution.

The scores flow through a judge, the taxonomy through single runs, the headline through an inclusion criterion — and the internet was on.

- **The judge has never been shown a human.** Every number flows through **one LLM judge (GPT-5.4)**; the only check is against *two other LLMs* ($\kappa = 1.0$, binary labels). Perfect agreement is *suspicious*, not reassuring — and the default judge is the **same family** as the leaderboard winner (GPT-5.5).
- **The 54% failure taxonomy is one run per cell.** *One run per (model, task)*, 226 model-failure runs — no repeats, no error bar, in a paper whose own results show agent runs are enormously noisy.
- **The headline hint result is near-circular.** Solvability-given-the-playbook was an *inclusion criterion* (Gate 4). The benchmark contains, by construction, tasks solvable with the playbook. And “80/82” is an *oracle* across many runs; the best single agent is 51.6%.

And the internet leaked. Gemini CLI found “*preliminary results... released on the Scale Labs Blog*” and returned the answer — and it is **#2 at 50.0%**. Excluded from *one* qualitative table; the aggregate is never audited.

Part I — The papers

1. Context: can an agent do a scientist's analysis?
2. GeneBench-Pro — simulate, so you can grade judgement
3. DrugDiscoveryBench — real tasks, an expert's playbook
4. How these agents fail — and what each failure demands

Part II — The design

5. The design: an LLM, a loop, three ledgers, a gate
6. How would I know it works?

Over half of failures are not coding failures

DDB classified 226 failing runs into root causes. This table is what a builder actually needs — and each row earns a mechanism in Part 2.

Failure mode	Share	Their definition
Domain reasoning	54.0%	“applies an incorrect scientific premise or misinterprets the data it has, even though its inputs and tools are correct ”
Derivation error	18.6%	right approach, wrong calculation
Retrieval	16.4%	wrong source, or never reads a provided file
Constraint	7.5%	violates an explicit instruction in the prompt
Final-answer slip	3.5%	every step right, reports the answer wrongly

DrugDiscoveryBench, Table 3 — $n = 226$ failing runs

Read the top row again

Over half of all failures are not coding failures. The tools worked. The Python ran. The data was right there. The next five slides make each failure concrete — and name the Part 2 mechanism it earns.

Failure 1 — it notices, and then does nothing about it

GBP's central diagnosis. If you remember one thing from Part 1, remember this.

*“models often complete substantial portions of the workflow but exhibit a consistent gap between **noticing** and **acting**. . . As a result, models often select wrong estimators or persist on initially plausible but incorrect analysis paths.”*

GeneBench-Pro, abstract, p. 1 — verbatim

The intuition, in one sentence

The model *sees* the problem. It runs `df.describe()`. It observes the sentinel values. It may even print a comment about them. **Then it cleans the column and proceeds with the analysis it had already decided on.**

The observation never reaches the decision.

→ **Part 2: a Findings Ledger** — *enforce the observation*. A noticed problem becomes an open obligation, and the exit is blocked until it is discharged.

Failure 2 — the question quietly degrades

The one I find most alarming, because nothing crashes and the arithmetic is correct.

DDB — the melanoma task

Asked: which gene is the melanoma marker?

Mid-trajectory the qualifier “*melanoma*” silently falls out. The ranking becomes a *gene-wide* variant count. **PTEN** wins on sheer volume. (Truth: CDKN2A.)

GBP — a denominator trap

The estimand is a probability over **the entire roster**. The agent standardises over **tested partners only** — a different denominator, a plausible answer, and the wrong one.

The pattern

A constraint or a population silently drops out of the question partway through, and every step after that is a correct analysis of the *wrong question*. GBP names the end state exactly: “*A statistically valid final model is applied to the wrong data or population, on the wrong scale, or on the wrong conceptual level.*” (p. 6)

→ **Part 2: a Question Contract** — write the estimand, population, units and constraints down as structured state, and *re-show it every turn*, so decay becomes visible.

Failure 3 — the right method, applied to the wrong object

DDB — the net-charge task

The chemistry is **correct**. The method is **correct**. It is executed on the neutral **reference** molecule instead of the **bound ligand**. Answer: 0. Truth: -1.

The part that should worry anyone building a self-checking agent

The agent ran **three internal consistency checks**. “*All three independent checks agree*” — all three run **on the wrong molecule**.

Internal consistency is not validity.

A verifier that reads the agent's own narrative will *agree* with the agent's narrative. Verification has to come from *outside* the trajectory.

→ **Part 2: a fresh-context validator** — a *different* model that never sees the agent's reasoning, only the contract, the code, and what it printed.

Failure 4 — its own code printed the right number

My favourite failure in either paper, because it is so cheap to prevent.

“its own code printed the correct group-level count of 1, but the final tally used the atom-level count of 2... It reported 8 interactions instead of 7.”

DrugDiscoveryBench, p. 27 — verbatim

- The analysis was **right**. The code was **right**. The output on screen was **right**.
- **The number in the answer was not the number in the output.**

This is a 15-line regex away from being impossible

An LLM reviewer *might* catch this. A deterministic check — “every number you report must appear verbatim in the stdout of code that actually ran” — catches it **every time, for free**.

→ **Part 2: Evidence Grounding** — *ground the results*. Regex every reported number against real stdout; reject the answer if a number was never printed.

Failure 5 — nobody checks the answer at the end

*“The last chance to catch the slip is at the final answer: a human who had misread the task the same way would look at the result, recognize it as a meaningless response to the user’s actual goal, and backtrack. **None of the failing models caught this.**”*

DrugDiscoveryBench, p. 13 — verbatim

What that looks like in practice

Claude Opus answers **BRCA2** — a *breast*-cancer gene — to a **melanoma** question. Any domain expert glancing at that output would stop dead. **The agent submits it.**

One diagnosis, from two labs and two domains

*“the agents knew which database to query and how to compute the property... But somewhere along the execution they **drop a constraint, commit too early, fail to backtrack.**” (DDB, p. 12)*

The bottleneck is not code, not knowledge, not context length. It is **procedure** — and **procedure is something an engineer can build.**

→ **Part 2:** finishing is a **gated action** — an LLM judge must *approve* the draft before `submit_answer` is allowed to return.

Part II

The design

An LLM, a loop, and the gates on top —
and how I would know whether any of it worked

Part I — The papers

1. Context: can an agent do a scientist's analysis?
2. GeneBench-Pro — simulate, so you can grade judgement
3. DrugDiscoveryBench — real tasks, an expert's playbook
4. How these agents fail — and what each failure demands

Part II — The design

5. The design: an LLM, a loop, three ledgers, a gate
6. How would I know it works?

The assignment, and the thesis

Part 2, Option A

Give an agent a **question in English** and a **folder of data files**. It must explore the data, choose an approach, write and run code, **check its own intermediate results**, and return a **structured answer**.

The design thesis — straight out of Part 1

The base model **already writes competent pandas**. The interesting engineering is not “make it code”. It is **making it *keep the thread***.

Part 1 says the agents drop exactly **three** things. So make each one **explicit state the agent is not allowed to drop**:

What gets dropped	The failure	The mechanism
the question	the melanoma qualifier evaporates	Ledger 1 — Question Contract
the finding	notices the sentinel, ignores it	Ledger 2 — Findings Ledger
the number	printed 1, reported 2	Ledger 3 — Evidence Grounding

Do not spend the budget making the model smarter. Spend it making it **structurally impossible** to drop the thread.

The base — what an agent actually is

Strip the mystique. There are exactly three ideas, and all three are small.

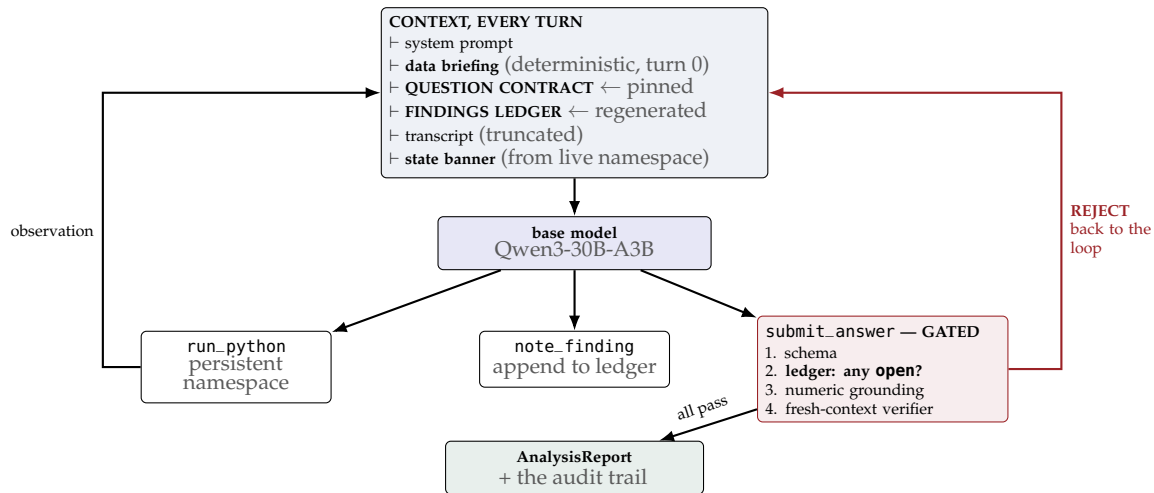
- ❶ **The LLM is a pure function.** messages $\rightarrow$ message. No memory, no state, no ability to act. Everything else is something *you* build around it.
- ❷ **Tool calling is not the model running code.** The model emits a *structured request* — “please call `run_python` with this string”. **Your program executes it** and appends the result to the transcript. The model never touches the machine.
- ❸ **The agent loop is a while loop.** (ReAct, Yao et al.)

```
while step < budget:
    msg = llm(transcript, tools)          # think
    if msg.tool_calls:
        result = execute(msg.tool_calls) # act
        transcript.append(result)        # observe
    if submitted: break
```

That is the whole agent — about 40 lines

Everything interesting in this design is in what surrounds that loop, and every piece of it was earned from a failure in Part 1.

The architecture



The loop is unremarkable, and that is the point. **All of the design is the box on the right and the two ledgers pinned into the box on top.**

Three tools — and why not more

- `run_python(code)` — executes in a **persistent namespace**, returns `stdout` + truncated traceback. Variables survive across steps, exactly like Jupyter cells.

One general tool beats N specific ones. Anything pandas can do, the agent can do — no capping it at whatever verbs I anticipated.

- `note_finding(...)` — the ledger.
- `submit_answer(...)` — the **gated** exit.

That is all

Every additional tool is **surface area to misuse** — and Part 1 shows the models are **not failing for lack of affordances**.

DDB gave its agents **226 functions**. They still scored 51.6%, and 54% of the failures happened with *correct inputs and correct tools*.

The single most important context rule

The model never sees the data.

It sees **metadata, summaries, and the output of code it wrote**.

This is what makes the design **scale-invariant**: a 200,000-row dataframe and a 200-row one cost the *same* tokens, because in both cases the model looks at `df.dtypes`, `df.head(3)`, and whatever it chose to print. The dataframe lives in the executor process.

Addition 0 — the detector, and the idea the design rests on

Before the model is called even once, plain Python looks at the data.

A deterministic profiler runs at turn 0

Shape, dtypes, head(3), null counts, cardinality — *and it runs detectors*: sentinel values, duplicate identifiers, numeric-looking strings, group imbalance.

Then the move that matters. What it finds is **not printed as a helpful note**.

It is **registered as an *open obligation*** — before turn one.
Information can be ignored. An obligation cannot.

The division of labour the whole design rests on

deterministic code	finds the mechanical problems, and makes them un-ignorable
the model	decides what they mean for <i>this</i> question

It is not a straitjacket: on the response-rate task the agent looks at the pre-seeded -999 finding and **dismisses** it — correctly, because that column never enters the calculation. **We do not force a conclusion. We force the observation to *reach* the decision.**

Ledger 1 — the Question Contract

Attacks Failure 2 — the melanoma scope-drop; DDB Constraint (7.5%); GBP's "wrong population / wrong scale".

```
class QuestionContract(BaseModel):  
    estimand: str # the exact quantity  
    population: str # WHICH ROWS. the denominator.  
    units: str # percent? fraction? log?  
    constraints: list[str] # every stated filter  
    premises: list[str] # what the QUESTION  
                    # assumes -- must check  
    question_is_precise: bool # REQUIRED  
    ambiguities: list[str]
```

Pinned into context **every turn**. Shown to the verifier at submit time.

Example — a filled contract

```
estimand: mean biomarker change, baseline→wk 12  
population: arm A only, completers  
units: raw assay units (not %)  
premises: ["arm A was randomised"]  
question_is_precise: true
```

Two fields the *evaluation* demanded

premises — a question can be *wrong*. “Which of the **four** sites had the highest response rate?” — there are **three**. “**Why** do women respond better?” — **they don’t**.

Without this field the agent answered both, *fluently*, laundering a false premise into a fact. 0/3 → 3/3.

question_is_precise — a *required boolean*, and that is the whole trick. With an optional ambiguities list the agent left it empty **every single time**.

A field the model *may* leave empty is a field the model *will* leave empty.

Ledger 2 — the Findings Ledger

Attacks Failure 1 — the notice–act gap directly: GBP’s headline, and DDB’s 54% domain-reasoning bucket.

Every time the agent notices *anything*, it must log it:

```
note_finding(  
  observation = "biomarker_x has 47 values "  
               "of exactly -999.0",  
  implication = "missing-data sentinels, not "  
               "measurements; drag mean ~30%",  
  status      = "open", # open|acted|dismissed  
)
```

The rule that makes it bite

submit_answer is **hard-blocked** while *any* finding is open.

To close one: **act** (point at the code step) or **dismiss** (say why it doesn’t affect the estimand). The implication field is where the *propagation* happens — the step both papers watch it skip.

Example — a run’s ledger, the audit artifact

status	what it noticed → what it did
ACTED	biomarker_baseline has 88 -999s → excluded before any mean
ACTED	48 patient_ids appear twice → kept the last row per patient
ACTED	arm imbalanced on severity → stratified; conditional effect
DISMISS	batch B on a shifted scale → never enters a rate; no effect

One is an **oracle**. The other is a **colleague**. ~40 lines, one tool.

Ledger 3 — Evidence Grounding

Attacks Failure 4 — DDB's Derivation errors (18.6%) and Final-answer slips (3.5%).

The rule, stated in the system prompt as an absolute

Every number you report must have been printed by code you ran.
Estimating is not a permitted operation.

And then it is **enforced deterministically**, not requested politely:

- Regex the numbers out of the answer's evidence list.
- Normalise — strip commas and percent signs, compare at 4 significant figures, **also test** $\times 100$ **and** $\div 100$ for the percent-vs-fraction confusion.
- Require set containment against the **concatenated stdout of the whole run**.

REJECTED: the value 0.62 **in** your evidence never appeared **in any** executed output. Compute **and print** it before citing it.

No LLM is involved. It cannot be talked out of it.

DDB documents this exact failure *in a frontier model* — “printed 1, reported 2”. An LLM reviewer *might* catch that. **A regex catches it every time, for free, in fifteen lines.**

The gated exit — finishing is an *action*, not a default

Attacks Failures 3 and 5 — “the last chance to catch the slip is at the final answer. None of the failing models caught this.”

submit_answer runs four gates **in order — cheapest and most deterministic first**:

	Gate	Rejects when
1	schema	pydantic fails → the ValidationError text goes back and the model self-repairs
2	ledger	<i>any</i> finding is still open
3	grounding	a reported number never appeared in real output
4	verifier	one LLM call — and only if 1–3 passed

Never pay for an LLM call to catch what a regex would have caught.

The verifier’s critical design detail

It is a **different model family** (self-preference bias — Zheng et al. 2023), and: **it does not see the agent’s reasoning**. It sees only the **Question Contract**, the **data briefing**, the **code that ran and what it printed**, and the **draft answer**. Show it the chain of thought and it gets **anchored by the agent’s own narrative and rubber-stamps**. The **fresh context is the mechanism** — recall the three self-checks, all agreeing, all on the wrong molecule.

Everything, in one table

The failure (Part 1)	Where it comes from	What I add
It notices and does not act	GBP notice-act gap ; DDB domain reasoning 54%	Findings Ledger — a noticed problem is an open obligation ; the exit is blocked until it is discharged
The question degrades	DDB constraint 7.5%; the melanoma case	Question Contract — estimand, population, units, premises; pinned every turn
Printed 1, reported 2	DDB derivation 18.6% + final-answer slip 3.5%	Evidence Grounding — every number must appear verbatim in real stdout. A regex.
Nobody checks at the end	DDB: “none of the failing models caught this”	Fresh-context verifier — different family, never sees the reasoning
<i>It never noticed at all</i>	<i>neither paper measures this</i>	The detector — deterministic profiler pre-seeds the ledger before turn 1

Note the last row

It is the one mechanism **not** derived from the papers — it comes from a gap they leave: both diagnoses are established only on *frontier* models. **It will turn out to be the only one that matters.**

Part I — The papers

1. Context: can an agent do a scientist's analysis?
2. GeneBench-Pro — simulate, so you can grade judgement
3. DrugDiscoveryBench — real tasks, an expert's playbook
4. How these agents fail — and what each failure demands

Part II — The design

5. The design: an LLM, a loop, three ledgers, a gate
6. How would I know it works?

The evaluation is the design

Both supplied papers are, fundamentally, evaluation papers. So I steal their protocol — and say so.

How I built the benchmark — the simplest honest version

- 1 I **simulated two datasets** following GBP's principles, and **used a third, held-out** one.
 - 2 I asked **Claude to author 28 tasks** over them — lookups, aggregations, and *planted traps*.
 - 3 I grade with **binary, programmatic metrics**: ground truth is a pandas callable *only the grader* ever invokes.
- `penguins.csv` — real, clean, familiar. The agent should score 100%, and a guardrail should buy *nothing*.
 - `trial.csv` — **synthetic clinical data with a known data-generating process**, built so plausible-but-wrong analyses give *materially different* answers.
 - `sales.csv` — **a held-out domain**. E-commerce, not medicine. **Never designed against**.

GBP's third principle is the one people skip — and it saved me

For every task I *assert* the naive answer and the correct answer differ by **far more than the grading tolerance**. That guard **immediately killed two of my own tasks** — a median robust to the sentinels I planted, and an age question independent of its own filter. Both looked reasonable. Both graded nothing. **The assertion found them; I did not.**

The ablation study — switch off one mechanism at a time

4,480 runs · 28 tasks · 3 domains · 8 configs · 20 runs per cell · ~\$16 · 0 crashes · full agent 87%

Remove this	Pass	Δ vs full	Verdict
(nothing — the full agent)	87%	—	
the deterministic data briefing $\leftarrow$ the DETECTOR	60%	−27%	HURTS
<i>every guardrail at once</i>	67%	−20%	HURTS
the Findings Ledger $\leftarrow$ my centrepiece	81%	−6%	HURTS
the fresh-context verifier	87%	−0%	no detectable effect
observation truncation	87%	+0%	no detectable effect
the numeric grounding gate	87%	−0%	no detectable effect
the Question Contract	88%	+1%	no detectable effect

Paired hierarchical bootstrap vs the full agent, 10,000 resamples (GBP's scheme). On *trap* tasks the guardrail stack still cuts the wrong-attractor rate 24% $\rightarrow$ 7% — a 3.3 $\times$ reduction.

Removing *twenty lines of pandas* hurts more than removing *every gate combined*

I built this design around **gates**. The thing carrying it is the **detector** — the mechanism I spent the *least* time on.

A gate is only as good as the detector feeding it.

The gates were not *wrong*. They were **redundant** — the detector in front of them was already doing the job. Remove the detector and the gates have nothing to gate.

The objection I want to raise before you do

GBP says the opposite: *frontier models consistently notice* — the headroom is in *acting*. **Because we are not talking about the same model.**

GBP's subjects are **GPT-5.6-class**. My agent is **Qwen3-30B-A3B** — chosen *precisely because it is weak*. For a frontier model, `df.describe()` runs and the sentinel is spotted — noticing is the solved half. For a 30B model at \$0.10/1M, **noticing is genuinely not reliable**, so the profiler **substitutes for a capability the base model lacks**.

The notice–act gap is a frontier-model problem. The *notice* gap is a small-model problem.

Which half of the scaffolding earns its keep depends on which half of the job your base model already does for free — which is an argument for **owning an eval harness**, not a list of best practices. The right answer changes when you change the model.

The failure the average hid

'sales' scores 88% — better than the domain I designed against. That was the number I wanted. Now read the same runs per task.

	full agent	landed on the <i>documented naive answer</i>
t4_simpson — Simpson's paradox, medical (designed against)	40%	55%
s4_simpson_sales — Simpson's paradox, held-out domain	35%	50%
b3_ambiguous — "Did the biomarker improve?"	5%	—
25 easier tasks, spanning 75–100%	carry the 87%	

On the confounded comparison — the failure both papers were *written about* — my agent is worse than a coin flip

And when it fails it lands *exactly* on the confounded answer: it is **more likely to land on the naive answer (50–55%) than on the right one (35–40%)**.

An average is where a failure goes to hide.

And the fix is not another gate

There is **no detector for confounding**, so nothing is seeded, so the gate has nothing to gate. **A structural gate can force an observation to reach a decision — it cannot create one that was never made.** The roadmap is not more gates: it is **more detectors**.

The dashboard — and a live demo

```
http://localhost:8501
```

```
uv run streamlit run app.py
```

Tab 1 — Run the agent

- Ask a question, watch it work.
- **Read the audit trail** — what it noticed, and what it did about it.
- **Toggle the guardrails off, one at a time, and watch it fail.**
- An X-ray of every turn: what went into the model, what came out, how long it took.

Tab 2 — The evidence

- 4,480 runs, 8 ablations.
- The per-task table — **where the average hides the failure.**

A caveat I will state before you ask

The live demo serves **Qwen3-4B on a GPU I rent** (vLLM on Modal — the provider is a `base_url` and a key): **7× smaller** than the model the 4,480-run evaluation used, and the endpoint serves *one* model — so the verifier is **not** cross-family there. **Numbers produced live are not comparable to the evaluation**, and the dashboard says so itself.

What I would want you to take away

- ④ **Both papers agree, from opposite directions, on one diagnosis:** these agents fail by **losing the scientific thread** — not for lack of knowledge, tools, or compute.
- ② **So I spent the entire engineering budget on *procedure*:** turn the three things that get dropped — **the question, the finding, the number** — into state the agent is **not allowed to drop**.
- ⑥ **And then the evaluation told me I had built the wrong half.** The gates are redundant behind a **20-line detector** I barely thought about.
A gate is only as good as the detector feeding it.
- ④ **Which is a boundary condition on the papers, not a contradiction.** The notice–act gap is a **frontier-model** problem; the *notice* gap is a **small-model** problem — **and neither paper evaluates a single small model**.

Thank you. <http://localhost:8501>

Backup — per-task results (full agent, $n = 20$ each)

Task	Category	Pass	Naive
b3_ambiguous	ambiguous	5%	0%
s4_simpson_sales	trap:confounding	35%	50%
t4_simpson	trap:confounding	40%	55%
b2_false_premise_sex	false-premise	75%	0%
s3_refunds	trap:filter	75%	0%
t3_batch_units	trap:units	85%	10%
s13_ambiguous	ambiguous	85%	0%
s2_test_orders	trap:outliers	85%	0%
s10_holdout_rev_seg	trap:outliers	90%	0%
s11_holdout_conv	trap:constraint	90%	0%
s1_text_numbers	trap:dtype	90%	10%
s6_scope	trap:constraint	95%	0%
s9_false_premise	false-premise	95%	0%
h1_holdout_mean_severe	trap:sentinel	95%	0%

Task	Category	Pass	Naive
p1_lookup	lookup	100%	0%
p2_aggregate	aggregation	100%	0%
p3_groupby	groupby	100%	0%
p4_correlation	correlation	100%	0%
s5_age_sentinel	trap:sentinel	100%	0%
s7_groupby	groupby	100%	0%
s8_lookup	lookup	100%	0%
s12_holdout_age	trap:sentinel	100%	0%
t1_sentinel	trap:sentinel	100%	0%
t2_duplicates	trap:duplicates	100%	0%
t5_scope	trap:constraint	100%	0%
b1_false_premise_sites	false-premise	100%	0%
h2_holdout_rate	trap:duplicates	100%	0%
h3_holdout_scope	trap:constraint	100%	0%

Read this table, not the headline. 25 tasks spanning 75–100% carry the 87%. The three red rows are the ones the papers were written about.

Backup — where the machinery works exactly as designed

t3_batch_units — assay batch B reporting $10\times$ too large

0/20 without the guardrails → **17/20** with them

- 1 The **detector** sees the batch on a shifted scale.
- 2 It **seeds the finding** as an open obligation — before turn one.
- 3 The **gate** will not let the agent submit until it has been discharged.

The whole thesis, in one task

Wherever a detector feeds it, the gate is worth 17 tasks out of 20.
Where no detector feeds it (confounding, ambiguity), the gate is worth nothing.

Backup — the instrument I built to stop fooling myself, lied

I criticised GBP's error bars in Part 1. Then I made the same mistake, and only caught it by accident.

I ran **the identical benchmark twice**, either side of a change I had **proved was inert**.

Removing the Findings Ledger	Δ	95% CI	Verdict
run A	-11.1%	$[-18.6, -4.3]$	"SIGNIFICANT"
run B	-1.8%	$[-8.9, +4.6]$	<i>"no detectable effect"</i>

Same code. Same tasks. Opposite conclusions. With **10 runs/cell** the bootstrap resamples *from the ten outcomes I happened to observe*: a 1/10 cell cannot reach the 60% the next run produced. **A bootstrap gives you the variance of the data you have**, and nothing about the data you did not collect.

So I doubled to 20 runs/cell — and report the *spread* between two halves

If a verdict flips between two runs of the same experiment, it was never a verdict. **It was weather.** Two of the seven mechanisms' signs flip; only the **briefing** (-27%, 4-pt spread) I defend without hedging.

Colophon — where the numbers come from

- **Every claim about the papers** was checked against the PDFs (pdftotext + a full re-read). A pasted “DrugDiscoveryBench” summary disagreed with the paper on several figures — Opus 4.8 max is 45.1% not 47.2; GPT-5.5 Codex ceiling is 43.9% not 45.1; the representative task is KRAS/THR58, not the SMO one — and the PDF won every time.
- **Every number about my system** comes from `uv run python -m evals.report (evals/results.jsonl, 4,480 runs)` — not from prose.
- **With one exception, where the script is the liar.** `report.py` prints a headline cost of \$3.75 — but 77% of rows are cache replays that record \$0. On billed runs only, an analysis costs \$0.004 and the grid cost ~\$16.

I nearly “corrected” the design doc *down* to \$3.75

... on the rule “*if the prose and the script disagree, the script is right*”. Right for every **rate** in this project; **wrong for a sum** — because a **cache deflates a sum and cannot deflate a proportion**. “Trust the script” is itself an instrument, and it also **needs checking**.